assignment3

April 10, 2021

0.0.1 CS4423 - Networks

Prof. Götz Pfeiffer School of Mathematics, Statistics and Applied Mathematics NUI Galway

Name: Cian Mac Liatháin

ID: 11476078

1 Assignment 3

Provide answers to the problems in the boxes provided.

The buttons at the top of the page can be used to **create more boxes if needed**. The type of box can be changed from `Code` to `Markdown`. `Code` boxes take (and execute) `python` code. `Markdown` boxes take (and format nicely) **text input**. In this way, you can provide answers, ask questions, or raise issues, in words.

Some marks will be awarded for participation and engagement.

When finished, print this notebook into a **pdf** file and submit this to **blackboard**.

Deadline is Monday, April 12, at 5pm.

1.1 Setup

This is a `jupyter` notebook. You can open and interact with the notebook through one of sites recommended at its github repository.

Or, you can install and use `jupyter` as a `python` package on your own laptop or PC.

The following command loads the `networkx` package into the current session.
The next command specifies some standard options that can be useful for drawing graphs.

In order to execute the code in a box, use the mouse or arrow keys to highlight the box and then press SHIFT-RETURN.

[1]:
```
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
opts = { "with_labels": True, "node_color": 'y' }
opts2 = { "with_labels": True, "node_color": 'm' }
```

Should it ever happen that the notebook becomes unusable, start again with a fresh copy.

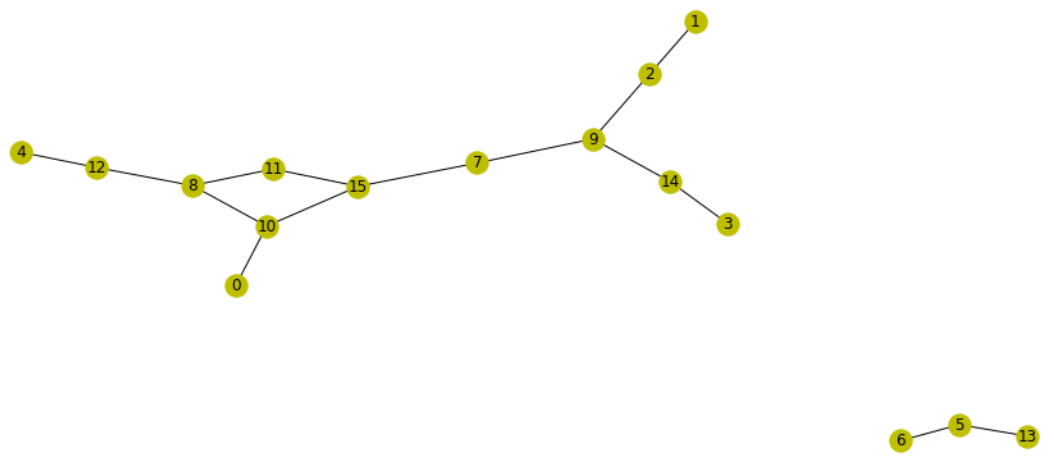
1.2 1. ER Model A

Definition (ER Model A: Uniform Random Graphs). Let $n \geq 1$, let $N = \binom{n}{2}$ and let $0 \leq m \leq N$.

The model $G(n, m)$ consists of the ensemble of graphs G with n nodes $X = \{0, 1, \ldots, n-1\}$, and m randomly selected edges, chosen uniformly from the N possible edges.

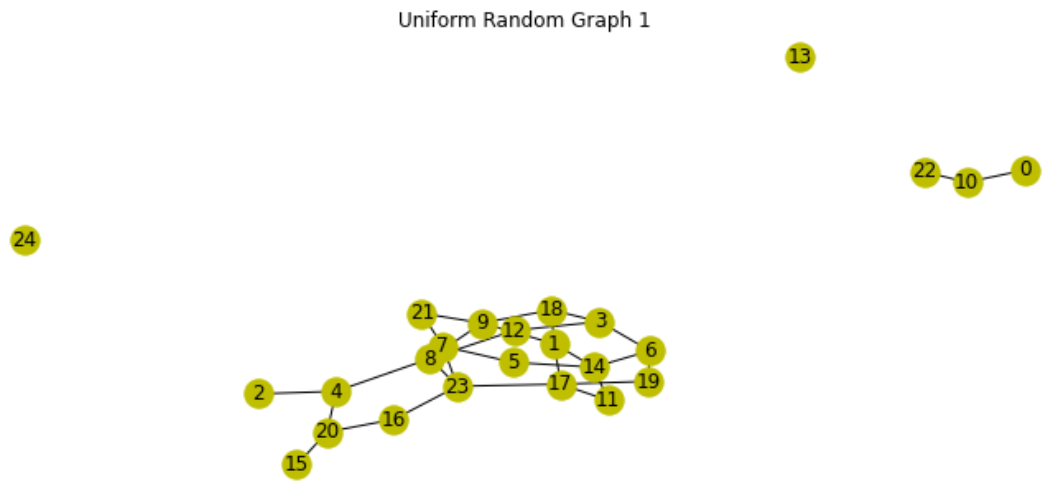
Model A random graphs in `networkx` can be generated with the function `nx.gnm_random_graph(n, m)`, where parameter n gives the number of nodes and parameter m the (exact) number of edges of the graph.

[2]:
```
plt.figure(figsize=(12,5))
G = nx.gnm_random_graph(16, 15)
nx.draw(G, **opts)
```

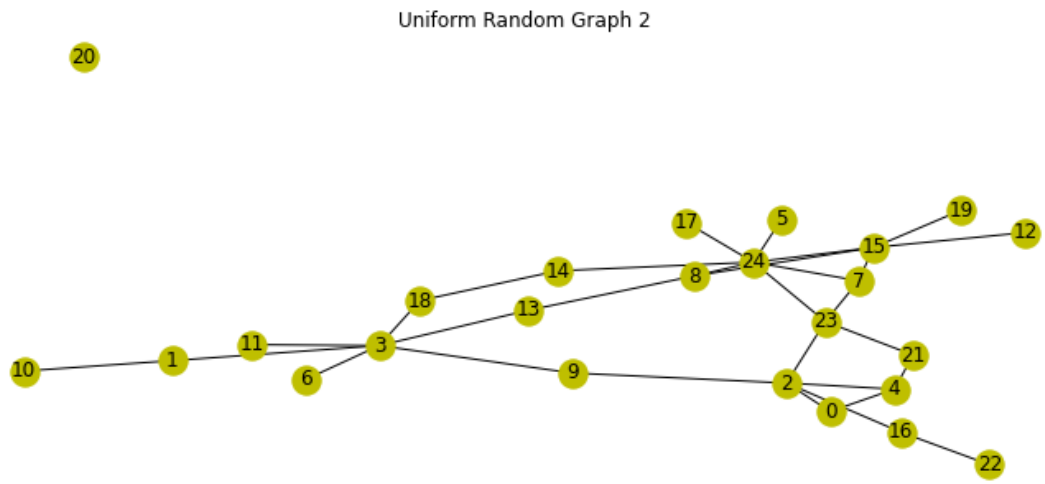


- Draw 6 random graphs drawn from model A with $n = 25$ nodes and $m = 30$ edges.
- For each graph drawn, determine its clustering coefficient C and its transitivity T.

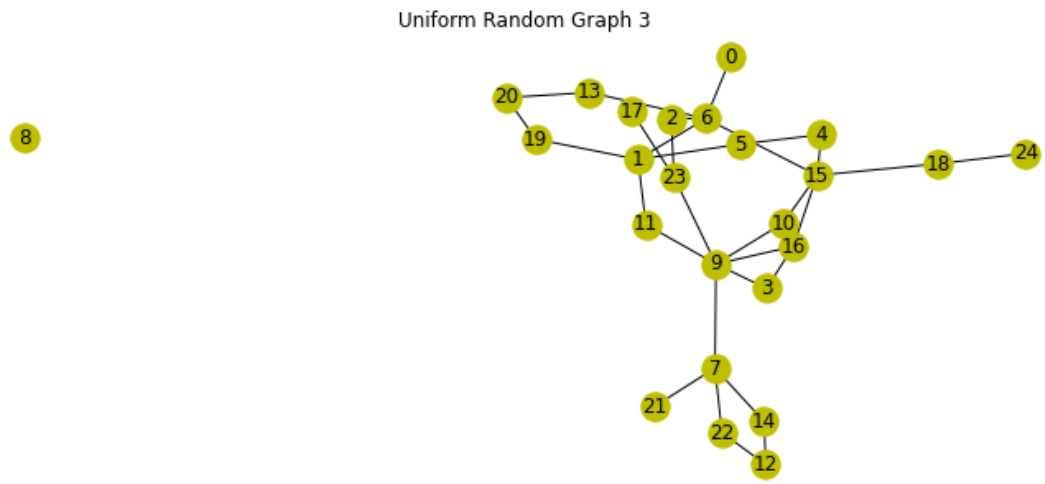
[89]:
```
graphs = []
for i in range(6):
    plt.figure(figsize=(12,5))
    G = nx.gnm_random_graph(25, 30)
    graphs.append(G)
    ax = plt.gca()
    ax.set_title('Uniform Random Graph ' + str(i+1))
    nx.draw(G, **opts)
    plt.show()
    print("Graph Clustering Coefficient: ", nx.average_clustering(G))
    print("Transitivity: ", nx.transitivity(G))
```



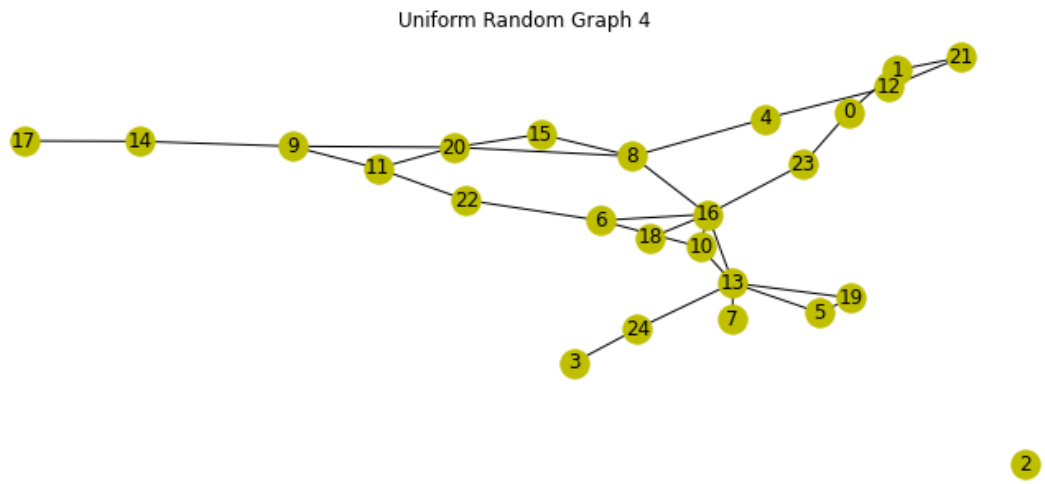

```
Graph Clustering Coefficient:  0.026666666666666665
Transitivity:  0.04918032786885246
```



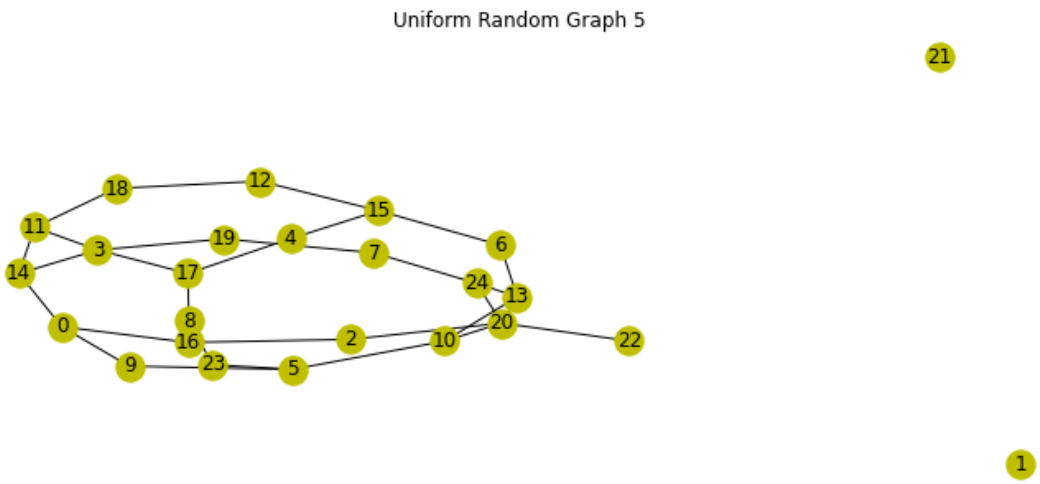

```
Graph Clustering Coefficient:  0.11771428571428572
Transitivity:  0.1518987341772152
```



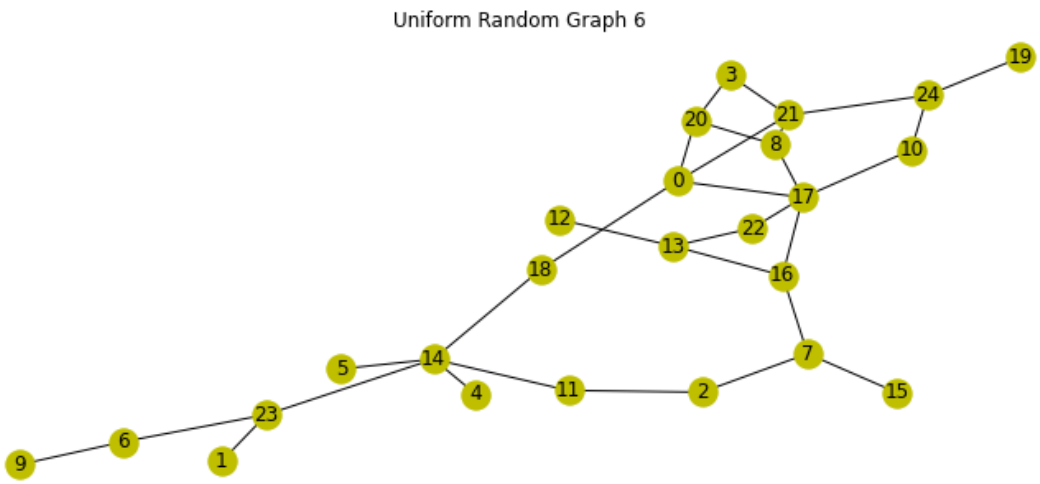

```
Graph Clustering Coefficient:  0.055999999999999994
Transitivity:  0.045454545454545456
```




```
Graph Clustering Coefficient:  0.21733333333333335
Transitivity:  0.22727272727272727
```




```
Graph Clustering Coefficient:  0.08133333333333333
Transitivity:  0.15789473684210525
```




```
Graph Clustering Coefficient:  0.0
Transitivity:  0
```

1.3 2. ER Model B

Definition (ER Model B: Binomial Random Graphs). Let $n \geq 1$, let $N = \binom{n}{2}$ and let $0 \leq p \leq 1$.

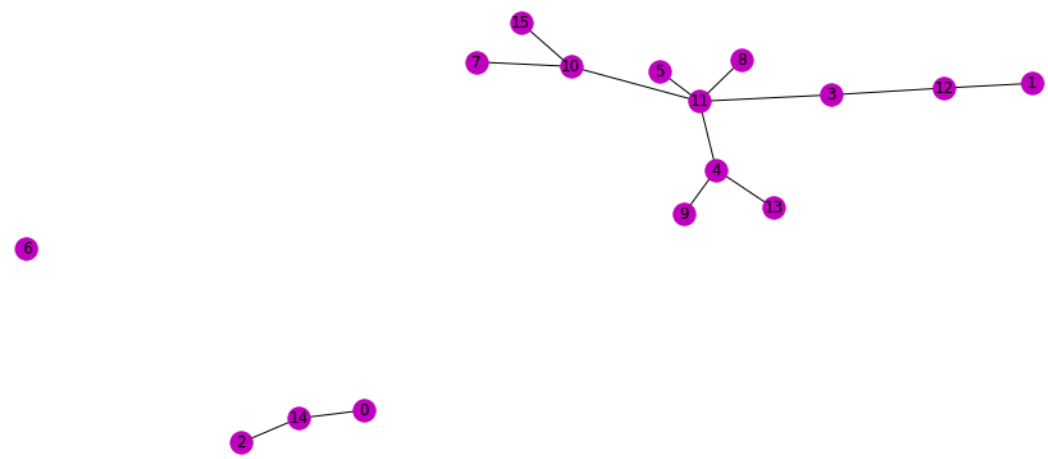
The model $G(n, p)$ consists of the ensemble of graphs G with n nodes $X = \{0, 1, \ldots, n-1\}$, and each of the N possible edges chosen with probability p.

Model B random graphs in `networkx` can be generated with the function `nx.gnp_random_graph(n,`

p), where parameter n gives the number of nodes and parameter $p \in [0, 1]$ the edge probability.

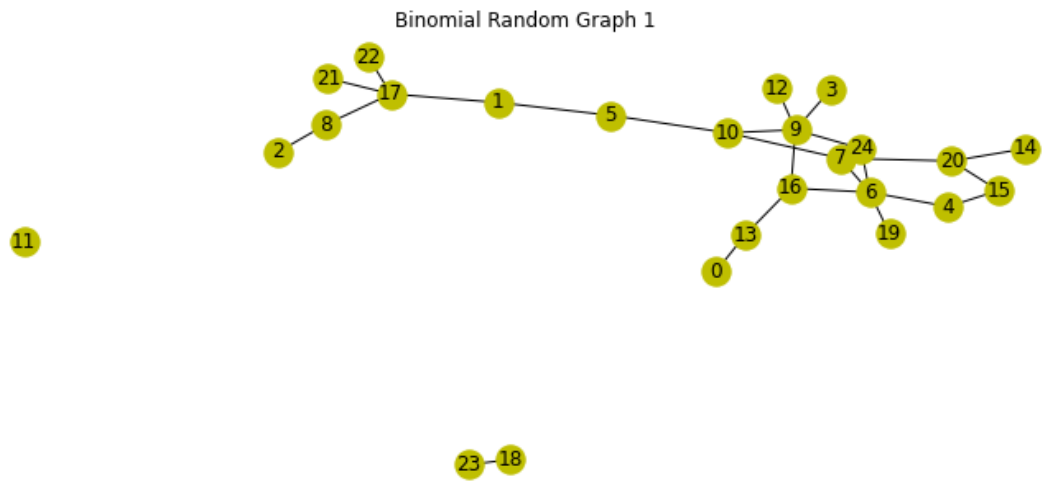
[4]:
```
plt.figure(figsize=(12,5))
G = nx.gnp_random_graph(16, 0.125)
print("#edges: ", G.number_of_edges())
nx.draw(G, **opts2)
```

```
#edges:  13
```

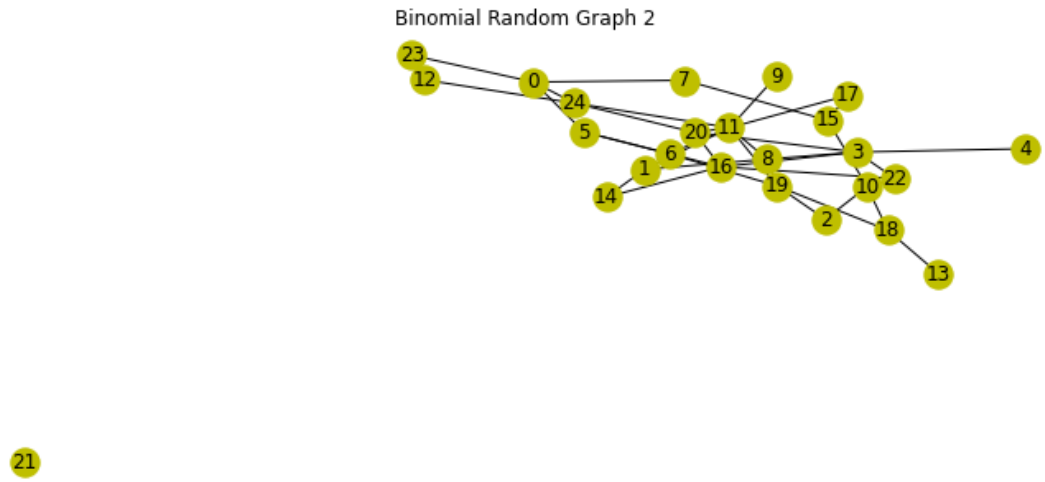


- Draw 6 random graphs drawn from model B with $n = 25$ nodes and edge probability $p = 30/\binom{25}{2}$.
- For each graph drawn, determine its number of edges, its clustering coefficient C and its transitivity T.

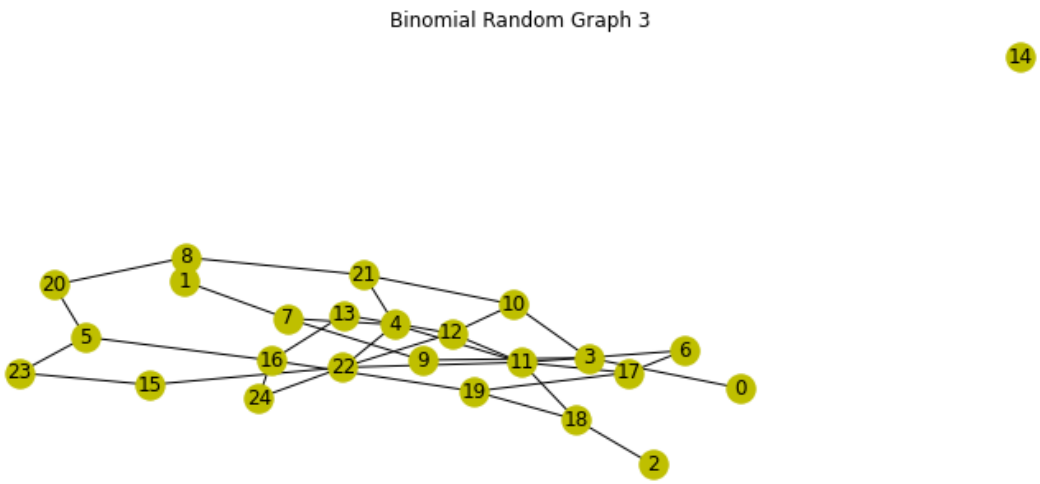
[90]:
```
graphs = []
for i in range(6):
    G = nx.gnp_random_graph(25, 0.1)
    graphs.append(G)
    plt.figure(figsize=(12,5))
    ax = plt.gca()
    ax.set_title('Binomial Random Graph ' + str(i+1))
    nx.draw(G, **opts)
    plt.show()
    print("Edges: " + str(len(G.edges())))
    print("Graph Clustering Coefficient: ", nx.average_clustering(G))
    print("Transitivity: ", nx.transitivity(G))
```



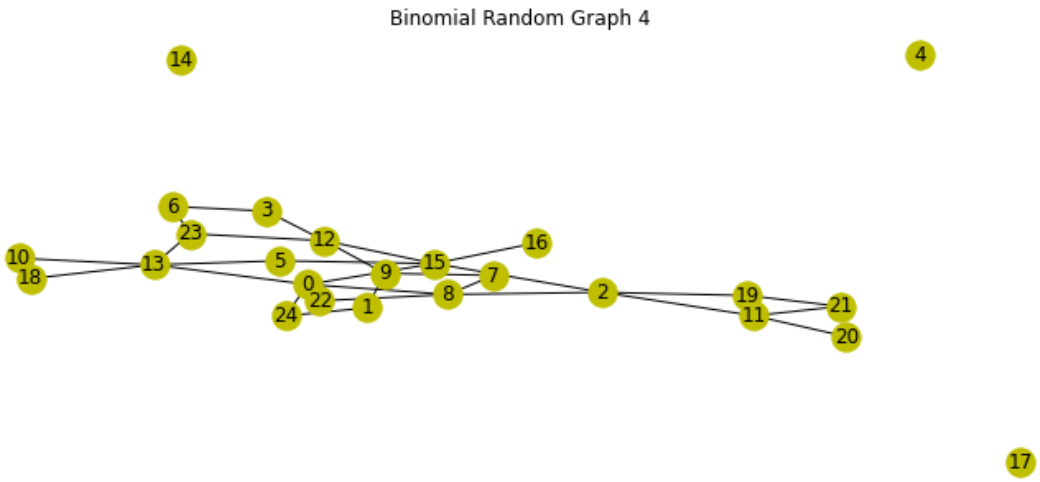

```
Edges: 25
Graph Clustering Coefficient:  0.0
Transitivity:  0
```



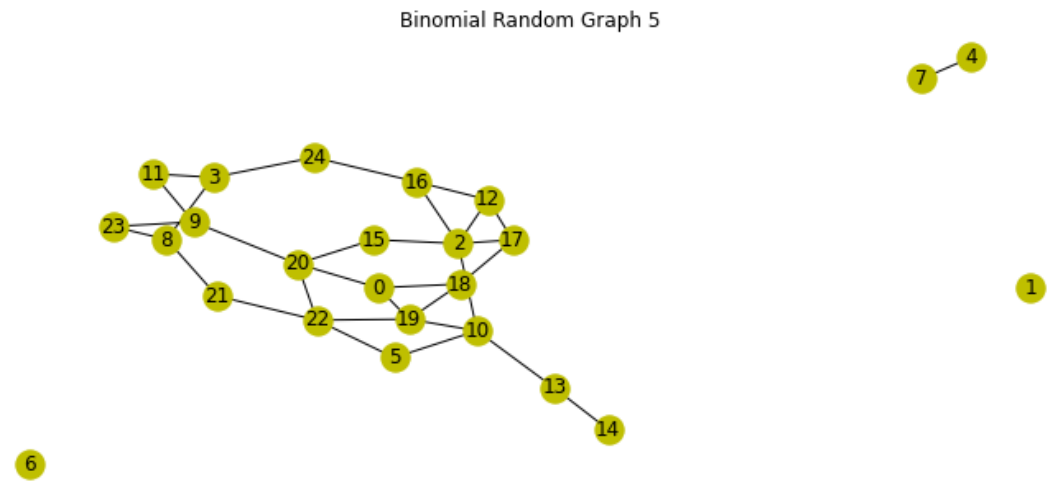

```
Edges: 38
Graph Clustering Coefficient:  0.10304761904761904
Transitivity:  0.16071428571428573
```



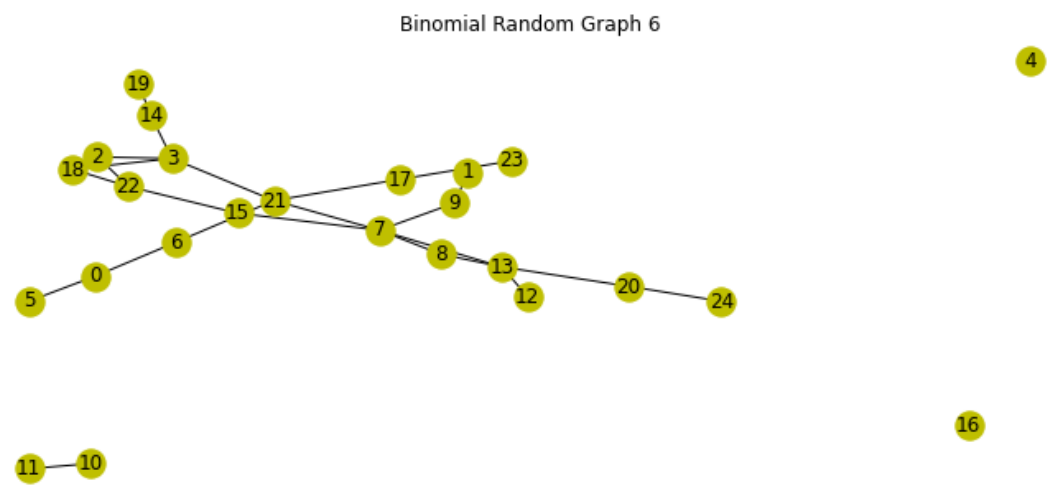

```
Edges: 34
Graph Clustering Coefficient:  0.08266666666666668
Transitivity:  0.11538461538461539
```




```
Edges: 28
Graph Clustering Coefficient:  0.0
Transitivity:  0
```




```
Edges: 31
Graph Clustering Coefficient:  0.09466666666666665
Transitivity:  0.13846153846153847
```




```
Edges: 24
Graph Clustering Coefficient:  0.05066666666666667
Transitivity:  0.07317073170731707
```

1.4 3. Degree Distribution

The **degree distribution** of a graph $G = (X, E)$ is the probability distribution of the node degrees of the graph G, i.e. the function $p\colon \mathbb{N}_0 \to \mathbb{R}$ defined by

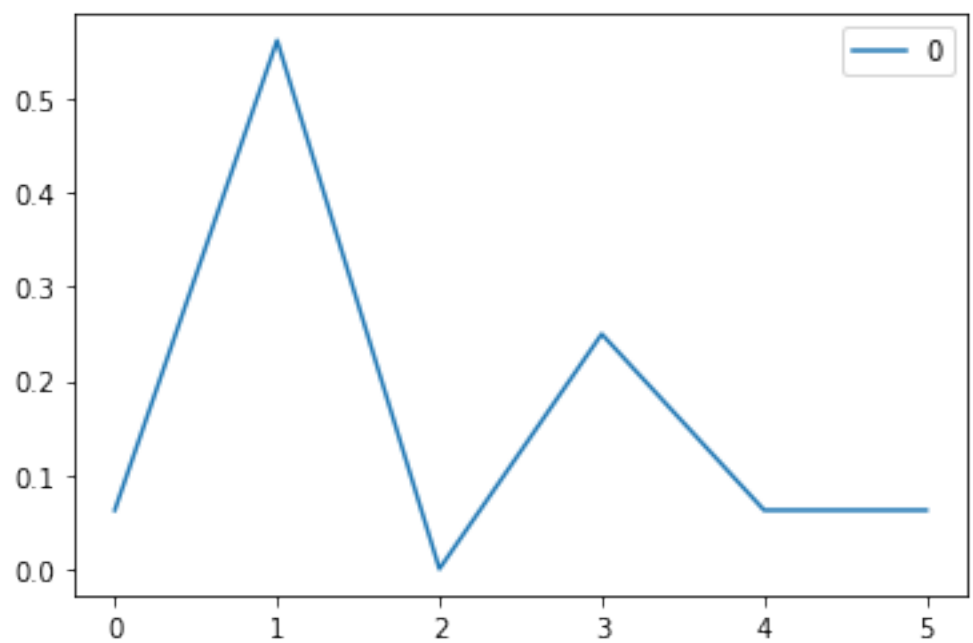
$$p_k = \frac{n_k}{n},$$

where $n = |X|$ is the total number of nodes in G, and n_k is the number of nodes of degree k. (Note hat $\sum_k p_k = 1$.)

In `networkx`, the numbers n_k can be determined by the function `nx.degree_histogram`. Then `python` list comprehension can be used to compute the numbers p_k from those. And those numbers, turned into a `pandas` dataframe, can be plotted nicely and quickly.

[29]:
```
G = nx.gnp_random_graph(16, 0.125)
n = G.number_of_nodes()
histogram = nx.degree_histogram(G)
distribution = [x/n for x in histogram]
df = pd.DataFrame(distribution)
df.plot()
```

[29]: <AxesSubplot:>



The degree distribution of a model B random graph is known to follow a **binomial distribution** $\mathrm{Bin}(n-1, p)$ of the form

$$p_k = \binom{n-1}{k} p^k (1-p)^{n-1-k}$$

Using the formula

$$\binom{n}{k} = \frac{n \cdot (n-1) \cdots (n-k+1)}{1 \cdot 2 \cdots k}$$

in `python`, the binomial coefficient $\binom{n}{k}$ can be computed with the following function:

[30]:
```
def binomial(n, k):
    prd, top, bot = 1, n, 1
    for i in range(k):
        prd = (prd * top) // bot
        top, bot = top - 1, bot + 1
    return prd
```

The binomial distribution $\mathrm{Bin}(n, p)$ can then be defined as:

[31]:
```
def b_dist(n, p, k):
    return binomial(n, k) * p**k * (1-p)**(n-k)
```

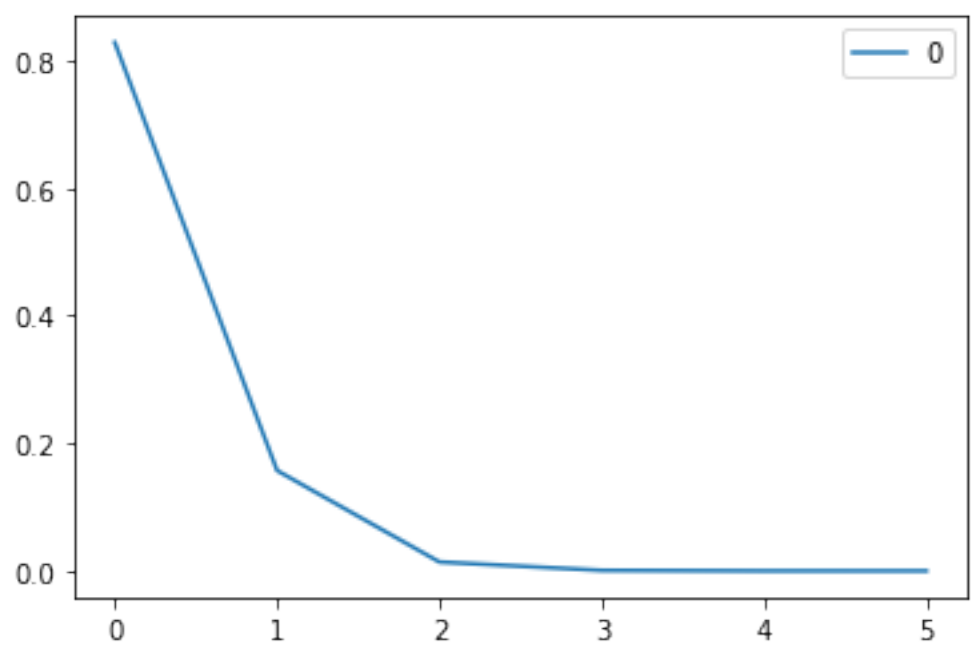
In order to compare the degree distribution of a random graph G on 16 points to the corresponding binomial distribution, one can compute and plot the values of $\mathrm{Bin}(16, p)$ for a suitable value of p, and k ranging from 0 to the highest node degree in G, as follows.

[32]:
```
len(histogram)
```

[32]: 6

[33]:
```
n, p = 16, 0.0125
bb = [b_dist(n-1, p, k) for k in range(len(histogram))]
df = pd.DataFrame(bb)
df.plot()
```

[33]: <AxesSubplot:>



In the limit $n \to \infty$ (keeping the expected average degree $p(n-1)$ constant), the binomial distribution $\mathrm{Bin}(n-1, p)$ is well approximated by the **Poisson distribution** defined by

$$p_k = e^{-\lambda} \frac{\lambda^k}{k!},$$

where $\lambda = p(n-1)$.

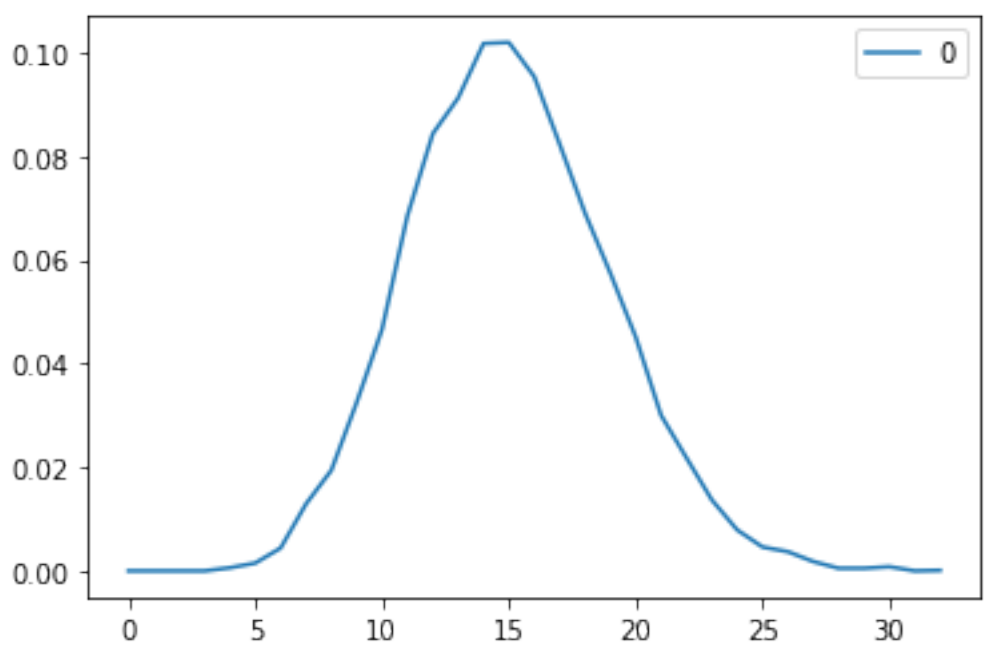
Using the functions `exp` and `factorial` from `python`'s `math` library, one can compute the Poisson distribution with the follwing `python` function:

[34]:
```
from math import exp, factorial
def p_dist(l, k):
    return exp(-l) * l**k / factorial(k)
```

- Create a model B random graph on $n = 10000$ points, with edge probability $p = 0.0015$, and plot its degree distribution. (This might take some time. If necessary, reduce the number of points …)

[88]:
```
G = nx.gnp_random_graph(10000, 0.0015)
n = G.number_of_nodes()
histogram = nx.degree_histogram(G)
distribution = [x/n for x in histogram]
df = pd.DataFrame(distribution)
df.plot()
```

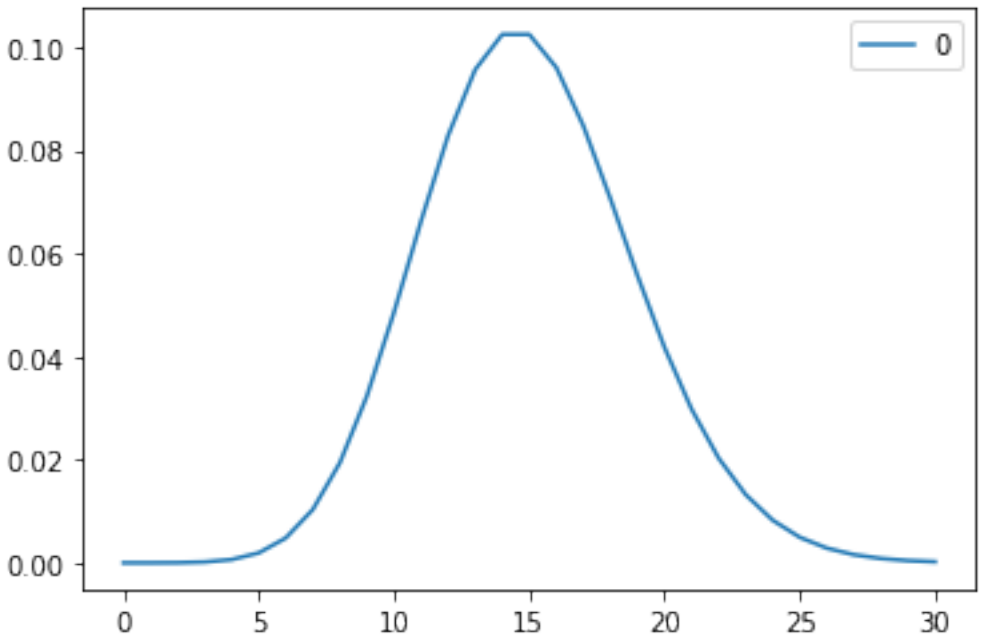
[88]: <AxesSubplot:>



- Compute and plot the binomial distribution that corresponds to a random model B graph on $n = 10000$ points with $p = 0.0015$ and k ranging from 0 to the highest node degree in G.

[82]:
```
n, p = 10000, 0.0015
bb = [b_dist(n-1, p, k) for k in range(len(histogram))]
df = pd.DataFrame(bb)
df.plot()
```

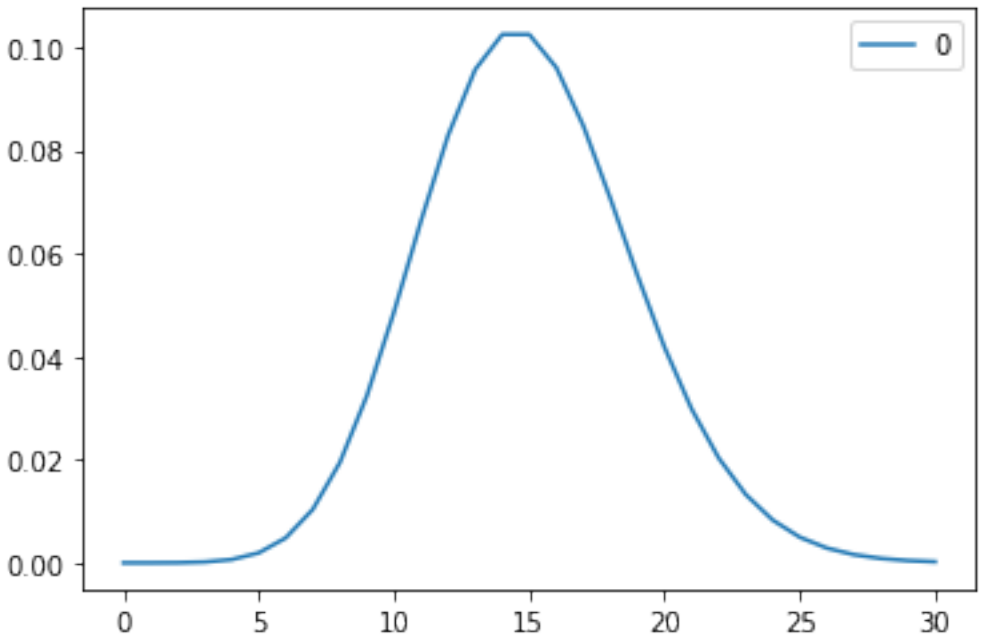
[82]: <AxesSubplot:>



- Compute and plot the Poisson distribution that corresponds to a random model B graph on $n = 10000$ points with $p = 0.0015$.

[83]:
```
n, p = 10000, 0.0015
l = p*(n-1)
pp = [p_dist(l, k) for k in range(len(histogram))]
df = pd.DataFrame(pp)
df.plot()
```

[83]: <AxesSubplot:>



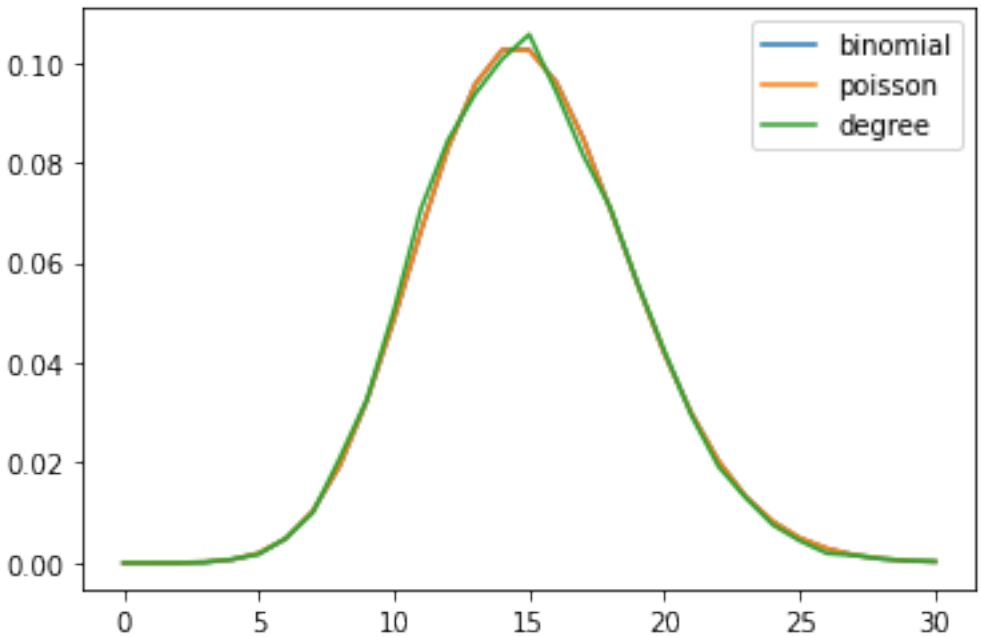
- What do you conclude from the plots? Can you plot all the distributions (binomial, Poisson, and the actual degree distribution of a graph G) into a single diagram?

We can see from the plots that the degree distribution closely follows a binomial distribution, and with a larger number of nodes we can see it also follows a poisson distribution.

This is even more apparent when plotting all three distributions in a single diagram:

[84]:
```
df = pd.DataFrame(index = range(len(histogram)))
df['binomial'] = bb
df['poisson'] = pp
df['degree'] = distribution
df.plot()
```

[84]: <AxesSubplot:>



1.5 4. Giant Components

Graphs of a certain relative size (number of edges in relation to number of nodes) have a **giant component**, a connected component that contains a substantial number of the nodes. For a random graph $G(n, p)$ in model B, the sudden appearance of a giant component can be described as a function of p (keeping n fixed), by measuring the size of the largest connected component.

With `networkx`, the connected components of a graph `G` can be found with the command `nx.connected_components(G)` and their respective sizes are determined with the command

```
sz = [len(c) for c in list(nx.connected_components(G))]
```

The `python` function `max` applied to a list of numbers finds the maximum number in the list, i.e., `max(sz)` is the maxium size in the list `sz`.

- For $n = 10000$, say, compute 50, say, random A-model graphs $G(n, m)$ for varying values of m, maybe equally spread out between 0 and 25000. (This might take some time. If necessary, reduce the number of points …)
- For each graph, determine the size of its largest connected component.
- Plot these sizes against the values of m.
- Can you spot a pattern? Where does the giant component start to grow? Which value of p does that correspond to?

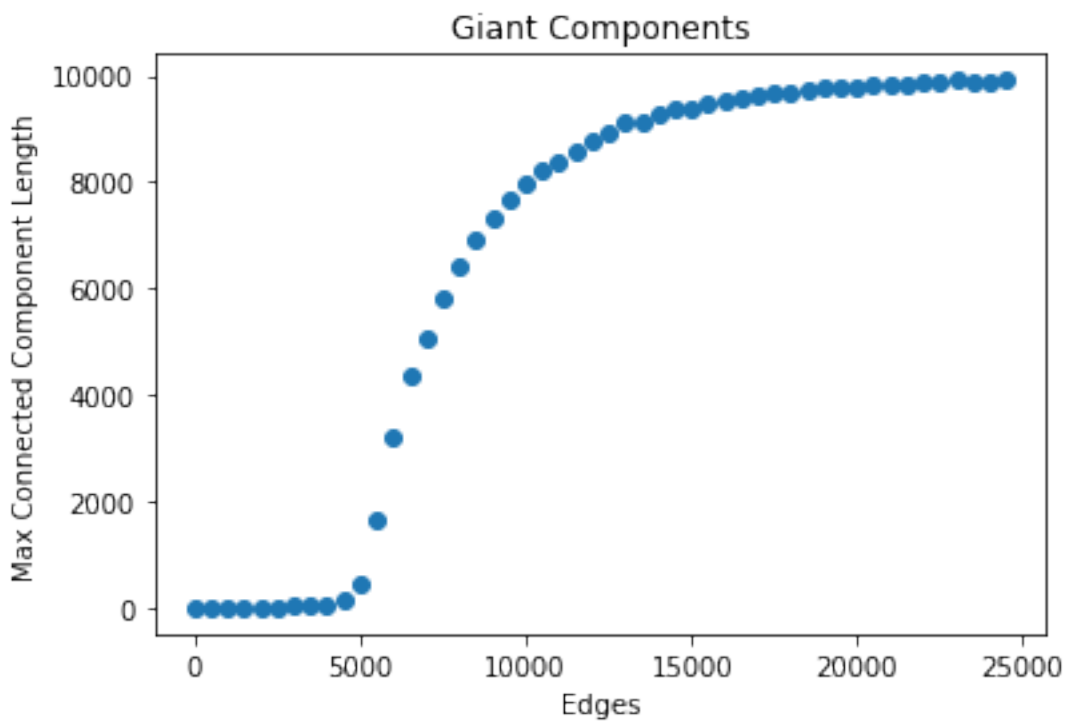
[73]:
```
edges = [500*m for m in range(50)]
cc = []
graphs = []
for i in range(50):
    G = nx.gnm_random_graph(10000, edges[i])
```

```
    graphs.append(G)
    cc.append(max(len(c) for c in nx.connected_components(G)))
```

[74]:
```
dictionary = dict(zip(edges, cc))
print("Edges: Largest Component")
print(dictionary)
```

```
Edges: Largest Component
{0: 1, 500: 4, 1000: 6, 1500: 8, 2000: 15, 2500: 17, 3000: 28, 3500: 42, 4000:
46, 4500: 154, 5000: 445, 5500: 1666, 6000: 3212, 6500: 4353, 7000: 5048, 7500:
5831, 8000: 6437, 8500: 6929, 9000: 7329, 9500: 7660, 10000: 7988, 10500: 8248,
11000: 8363, 11500: 8593, 12000: 8796, 12500: 8942, 13000: 9121, 13500: 9150,
14000: 9294, 14500: 9368, 15000: 9382, 15500: 9497, 16000: 9538, 16500: 9571,
17000: 9643, 17500: 9665, 18000: 9692, 18500: 9728, 19000: 9762, 19500: 9771,
20000: 9801, 20500: 9831, 21000: 9838, 21500: 9847, 22000: 9879, 22500: 9889,
23000: 9912, 23500: 9893, 24000: 9896, 24500: 9917}
```

[76]:
```
plt.scatter(edges, cc)
plt.title("Giant Components")
plt.xlabel("Edges")
plt.ylabel("Max Connected Component Length")
plt.show()
```



From the scatter plot we can see a pattern of logistic growth, where the giant component begins

growing in the 10th graph with 5000 edges, which is half the number of total nodes.

The size of the largest connected component in that graph is 445 which is $n^{2/3}$

[80]:
```
#p = average degree / nodes
# average degree = 2 * edges / nodes

average_degree = 2 * 5000 / 10000
p = average_degree / 10000
print("c:", average_degree)
print("p:", p)
```

```
c: 1.0
p: 0.0001
```